

Spring & Spring Boot, 원리 중심으로 이해하기

"코드를 외우지 말고, 요청 하나가 흘러가는 길과 스프링이 대신해 주는 일을 이해하자"

① 큰 그림

~5분

요청 한 번의 여정

② 핵심 원리

~10분

IoC · DI · 어노테이션

③ 스프링부트

~5분

무엇을 자동화했나

④ 실습 연결

~5분

자기소개서 서비스

⑤ 질문·정리

~5분

던질 질문 3개

① 큰 그림 — 요청 한 번의 여정

~5분 · 이 그림 하나로 시작

브라우저에서 자기소개서 상세보기(GET /intro/1)를 눌렀을 때 서버 안에서 벌어지는 일:



◀ **돌아오는 길** — 조회 결과(Intro 객체)가 되돌아오고, 템플릿(Thymeleaf)이 데이터를 끼워 **완성된 HTML**을 만들어 브라우저로 응답. 브라우저 입장에선 정적 페이지를 받을 때와 똑같이 "HTML을 받았을 뿐".

💡 이렇게 말해보세요

"여러분이 만든 자기소개 HTML은 **만들어 둔 파일을 그대로 주는 것**(정적)이었죠. 이제는 서버가 **DB 데이터를 가지고 그 순간에 HTML을 조립**합니다(동적). 스프링은 이 흐름을 **표준화해 둔 틀**이에요. 어느 회사의 어떤 스프링 프로젝트를 열어도 이 길은 똑같습니다."

왜 층을 나누나? — 식당 비유

층	식당으로 치면	역할	실습 파일
Controller	홀 직원	주문 받고 음식 내감 (요청/응답)	IntroController
Service	주방	레시피대로 조리 (업무 규칙)	IntroService
Repository	창고 담당	재료 꺼내고 채움 (DB 접근)	IntroRepository
Entity	재료(데이터)	층 사이를 오가는 데이터 한 건	Intro

→ 홀 직원이 직접 창고를 뒤지기 시작하면 식당이 커질수록 엉망이 된다 = 역할이 섞이면 코드가 커질수록 못 고친다.

1) IoC 컨테이너와 빈(Bein) — "객체를 스프링이 만들어 보관한다"

보통 자바 객체는 내가 new로 만듭니다. 스프링에서는 **주요 객체를 스프링이 시작 시점에 대신 만들어 보관함(컨테이너)에 넣고 관리합니다.** 그 객체들이 빈. 제어의 주도권이 나 → 스프링으로 뒤집혔다고 해서 **IoC**(제어의 역전).

💡 이렇게 말해보세요

"IoC 컨테이너는 **공용 도구함**이에요. 각자 드릴을 사는 게 아니라, 도구함의 드릴 하나를 필요한 사람에게 꺼내 주는 거죠. 왜? ① 하나만 만들어 모두가 공유 ② 객체끼리의 연결(조립)을 스프링이 대신 — 그 연결이 바로 DI입니다."

2) 의존성 주입(DI) — "필요한 객체를 스프링이 넣어준다"

```
// 스프링 없이 - 내가 직접 new
public class IntroController {
    private IntroService introService
        = new IntroService();
}
// 생성 방법을 내가 다 알아야 하고,
// 컨트롤러 10개면 서비스도 10개 생김
```

```
// 스프링 방식 - 생성자로 주입받기
@Controller
public class IntroController {
    private final IntroService introService;

    // "나 IntroService 필요해" 선언만 하면
    // 스프링이 빈을 찾아 넣어줌 = DI
    public IntroController(
        IntroService introService) {
        this.introService = introService;
    }
}
```

💡 이렇게 말해보세요

"내가 new로 만든 객체는 **스프링 컨테이너 밖의 존재**라 주입이 일어나지 않습니다. 필드 주입 코드라면 내부가 null이라 터지고, 생성자 주입이라면 아예 컴파일부터 막히죠. 규칙은 한 문장 — **주요 객체는 내가 new 하지 않고, 생성자로 받는다.**"

3) 어노테이션 — "@는 스프링에게 보내는 역할 선언"

어노테이션	스프링에게 전달되는 의미
@Controller	"웹 요청 받는 담당이야. 빈으로 등록해."
@Service	"업무 규칙 담당이야. 빈으로 등록해."
@GetMapping("/intro/{id}")	"이 URL의 GET 요청은 이 메서드가 처리해."
@Entity	"이 클래스는 DB 테이블과 짝이야." (JPA)

💡 이렇게 말해보세요

"어노테이션은 **실행되는 코드가 아니라 표지판**입니다. @Controller@Service는 스프링이 스캔해 빈으로 등록·연결하고, @GetMapping은 URL과 메서드를 잇고, @Entity는 JPA가 읽어 테이블과 매핑해요. 각자 담당이 표지판을 읽고 조립하는 것 — 그래서 스프링 코드는 '**역할 선언**'이 많습니다."

스프링 = 객체를 대신 만들고(IoC), 연결해 주고(DI), 역할은 선언으로(@) 읽는 틀

③ 스프링부트 — 스프링을 "바로 실행되게" 만든 것

~5분

가장 흔한 오해부터 바로잡기: 스프링부트는 스프링과 별개의 프레임워크가 아닙니다. 스프링을 쓰되, 시작을 무겁게 만들던 부분을 자동화한 도구입니다.

스프링만 쓰던 시절 (before)

- XML 설정 파일 수십~수백 줄 작성
- 라이브러리 버전 궁합을 사람이 맞춤
- 톰캣(웹서버)을 따로 설치하고 배포
- "화면에 Hello 띄우기"까지 만나절

스프링부트 (after)

- 자동 설정 — 관례대로 알아서, 필요한 것만 application.properties 몇 줄
- 스타터 의존성 — starter-web 한 줄에 호환 버전 묶음이 통째로
- 내장 톰캣 — main() 실행 = 서버 기동
- "Hello"까지 5분

이렇게 말해보세요

"부트(boot)라는 이름 그대로 '시동 걸어주는 것'이에요. 그래서 스프링부트를 배우면 스프링을 배운 것이고, 회사에서 쓰는 eGovFrame(전자정부 표준프레임워크)도 스프링 기반이라 지금 배우는 구조가 그대로 통합니다."

	스프링	스프링부트	eGovFrame (회사 실무)
정체	기반 프레임워크	스프링 자동화 도구	스프링 기반 공공 표준
화면/DB	자유	교육: Thymeleaf / JPA	주로 JSP / MyBatis
핵심 구조	Controller – Service – Repository(DAO) + IoC/DI/어노테이션 — 셋 다 동일		

④ 실습으로 연결 — 자기소개서 등록 서비스

~5분

실습 과제 "자기소개서 등록/조회 서비스" — 화면 기능 하나 = 컨트롤러 메서드 하나 = 표의 한 줄.

URL	하는 일	내부에서 벌어지는 일
GET /	목록 화면	findAll() → SELECT → 목록 HTML 조립
GET /intro/new	작성 폼	폼 HTML 반환
POST /intro	저장	save() → INSERT → 목록으로 redirect
GET /intro/{id}	상세 보기	findById() → SELECT WHERE → 상세 HTML

"save() 한 줄" 뒤에서 벌어지는 일 — DB 교육(JDBC)과 잇기

```
// 스프링이 내부에서 대신 하는 일
// (03_Database에서 직접 해보는 JDBC)
Connection conn = DriverManager
    .getConnection(URL, USER, PW);
PreparedStatement ps = conn.prepareStatement(
    "INSERT INTO intro ... VALUES (?, ?, ?)");
ps.setString(1, name); // 바인딩, 실행,
ps.executeUpdate();    // 자원 닫기, 예외 처리
```

```
// 스프링부트에서는
introRepository.save(intro);

// 접속·SQL 생성·바인딩·트랜잭션·자원 정리를
// 전부 스프링+JPA가 대신.
// 접속정보도 application.properties 3줄로.
```

이렇게 말해보세요

"save() 한 줄 뒤에서 스프링이 이만큼을 대신합니다. DB 교육(JDBC)에서 이 왼쪽 코드를 직접 겪어 보면 왜 프레임워크를 쓰는지 체감돼요 — 마법이 아니라 반복 작업을 맡기는 것. 원리를 아는 사람이 애러도 고칩니다."

Q1. "브라우저에서 등록 버튼을 누르면 무슨 일이 일어나죠?"

기대 답: 폼 데이터가 POST로 전송 → 컨트롤러가 받아 → 서비스 → 리포지토리 → INSERT → redirect → 목록 다시 조회. (①의 그림을 자기 말로 설명하면 합격)

Q2. "왜 서비스를 직접 new 하지 않고 생성자로 받나요?"

기대 답: 내가 new 한 객체는 스프링 컨테이너 밖이라 주입이 일어나지 않는다. 빈 하나를 공유하고, 연결은 스프링에 맡긴다.

Q3. "스프링부트가 스프링에 얹어준 것 3가지는?"

기대 답: 자동 설정 / 스타터 의존성 / 내장 톰캣. 보너스: "그래서 스프링부트 ≠ 별개 프레임워크"까지 말하면 완벽.

★ 한 장 요약 — 이 5문장만 남기기

- 1 웹서비스는 요청 → Controller → Service → Repository → DB → HTML 응답의 반복이다.
- 2 스프링은 객체를 대신 만들고(IoC/빈), 연결해 주는(DI) 틀이다 — 그래서 new 대신 생성자 주입.
- 3 @어노테이션은 역할 선언이다. 스프링이 그것을 읽고 조립한다.
- 4 스프링부트는 별개 프레임워크가 아니라 자동설정 + 스타터 + 내장톰캣으로 스프링에 시동을 걸어주는 도구다.
- 5 save() 한 줄 뒤에는 접속·SQL·트랜잭션·자원정리가 숨어 있다(03_Database의 JDBC 실습에서 직접 확인) — 원리를 아는 사람이 에러를 고친다.

면담 후 안내할 학습 경로

자료	내용
03_Database	DB 기본 · SQL · JDBC 실습 (스프링의 선수 과목)
Spring/SpringBoot	01 개념 → 02 구조 → 03 자기소개서 실습 → 04 헛갈림 사전 + 실행되는 완성 샘플
다음 단계	점프 투 스프링부트(무료 e북)로 게시판 확장 → eGovFrame 실무 코드 읽기

※ 이 자료의 그림·표는 사내 교육자료(intern-web-training)와 동일한 예제(자기소개서 서비스)를 사용합니다. 면담 중 "직접 확인해 보자"가 필요하면 Spring/SpringBoot/샘플/intro를 gradlew bootRun으로 띄워 보여주세요.